

```
import pandas as pd
import numpy as np
df=pd.read_csv('/content/penguins.csv')
```

```
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, f1_score
```

df.head()

	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	39.1	18.7	181.0	3750.0	MALE
1	39.5	17.4	186.0	3800.0	FEMALE
2	40.3	18.0	195.0	3250.0	FEMALE
3	NaN	NaN	NaN	NaN	NaN
4	36.7	19.3	193.0	3450.0	FEMALE

Next steps: [Generate code with df](#) [New interactive sheet](#)

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   culmen_length_mm      342 non-null    float64
1   culmen_depth_mm       342 non-null    float64
2   flipper_length_mm     342 non-null    float64
3   body_mass_g           342 non-null    float64
4   sex                   335 non-null    object
dtypes: float64(4), object(1)
memory usage: 13.6+ KB
```

```
for col in df.select_dtypes(include=np.number).columns:
    df[col] = df[col].fillna(df[col].mean())
for col in df.select_dtypes(include='object').columns:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```
#convert text into numbers(label encoder)
le_dict ={}

for col in df.select_dtypes(include='object').columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    le_dict[col] = le
```

df.head()

	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	39.10000	18.70000	181.00000	3750.000000	2
1	39.50000	17.40000	186.00000	3800.000000	1
2	40.30000	18.00000	195.00000	3250.000000	1
3	43.92193	17.15117	214.01462	4201.754386	2
4	36.70000	19.30000	193.00000	3450.000000	1

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
x=df.drop('sex', axis=1)
y=df['sex']
```

```
scaler = StandardScaler()
x_scaler = scaler.fit_transform(x)
```

```
#PCA (reduce dimension)

pca = PCA
#try different number of components
param_grid = {'n_components': list(range(1, x.shape[1] + 1))}

grid_pca = GridSearchCV(pca(), param_grid, cv=10)
grid_pca.fit(x_scaler)

best_n = grid_pca.best_params_['n_components']
print("Best number of components:", best_n)
```

Best number of components: 1

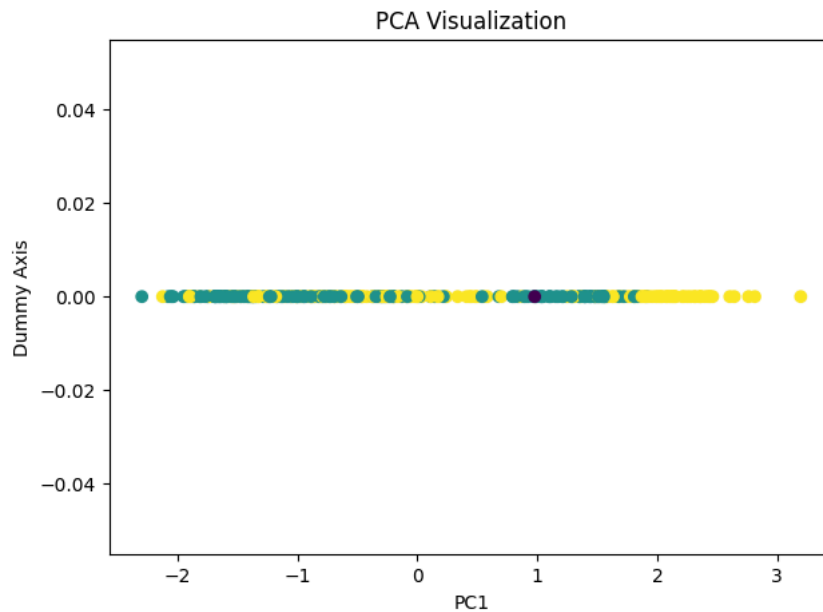
```
#apply PCA
pca = PCA(n_components=best_n)
x_pca = pca.fit_transform(x_scaler)
x_pca
```

```
[ 1.30237196e+00],
[ 2.13325442e+00],
[ 1.11061609e+00],
[ 2.35881907e+00],
[ 1.38984730e+00],
[ 1.30569343e+00],
[ 1.57652944e+00],
[ 1.68172941e+00],
[ 1.21237966e+00],
[ 2.08941681e+00],
[ 1.44840250e+00],
[ 2.43844333e+00],
[ 1.43098324e+00],
[ 1.75236699e+00],
[ 1.56129198e+00],
[ 2.31420731e+00],
[ 1.43885653e+00],
[ 1.89407222e+00],
[ 1.39914675e+00],
[ 1.82788191e+00],
[ 9.67617202e-01],
[ 2.42066040e+00],
[ 1.72381088e+00],
[ 1.99994788e+00],
[ 1.53224730e+00],
[ 1.86250464e+00],
[ 1.28486109e+00],
[ 2.03567383e+00],
[ 1.21087485e+00],
[ 1.90662633e+00],
[ 1.50868312e+00],
[ 1.50971100e+00],
[ 1.51214450e+00],
[ 1.77617286e+00],
[ 1.55585848e+00],
[ 2.44002623e+00],
[ 1.39412818e+00],
[ 2.15031297e+00],
[ 1.63144041e+00],
[ 1.62852203e+00],
[ 7.93709406e-01],
[ 2.39875149e+00],
[ 1.03705273e+00],
[ 2.01795642e+00],
[ 1.83755206e+00],
[ 2.36322210e+00],
[ 8.15739898e-01],
[ 2.07342722e+00],
[ 1.15424789e+00],
[ 2.81352970e+00],
[ 9.78205676e-01],
[ 2.2219459e+00],
[ 1.80929097e+00],
[-8.08667175e-16],
[ 1.55410864e+00],
[ 2.31304729e+00],
[ 1.54456934e+00],
[ 1.87259454e+00]]
```

```
#visualize PCA (first component only)

plt.figure(figsize=(7, 5))
#plot the first principal component against a dummy axis(eg, y=0)
#to visualize its distribution, colored by the target variable.
plt.scatter(x_pca[:, 0], y, c=y, s=50, edgecolor='k')
plt.xlabel('PC1')
plt.ylabel('Target Variable')
plt.title('PCA Scatter Plot')
plt.show()
```

```
plt.scatter(x_pca[:, 0], np.zeros(len(x_pca)), c=y)
plt.xlabel("PC1")
plt.ylabel("Dummy Axis")#updated label to reflect 1D visualization
plt.title("PCA Visualization")
plt.show()
```



```
x_train, x_test, y_train, y_test = train_test_split(
    x_pca, y, test_size=0.2, random_state=42
)
```

```
#decision tree with grid search
dt = DecisionTreeClassifier(random_state=42)

param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [2, 3, 4, 5, 6],
    'min_samples_split': [2, 3, 5, 10],
    'min_samples_leaf': [1, 2, 3, 4]
}

grid_dt = GridSearchCV(dt, param_grid, cv=10)
grid_dt.fit(x_train, y_train)

best_model = grid_dt.best_estimator_

print("Best parameters:", grid_dt.best_params_)
print("Best critrion:", grid_dt.best_params_['criterion'])
```

```
Best parameters: {'criterion': 'entropy', 'max_depth': 5, 'min_samples_leaf': 3, 'min_samples_split': 2}
Best critrion: entropy
```

```
y_pred = best_model.predict(x_test)
```

```
cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average='weighted')

print("Accuracy:", acc)
print("F1 Score:", f1)
```

```
Accuracy: 0.6521739130434783
F1 Score: 0.6427932418270582
```

